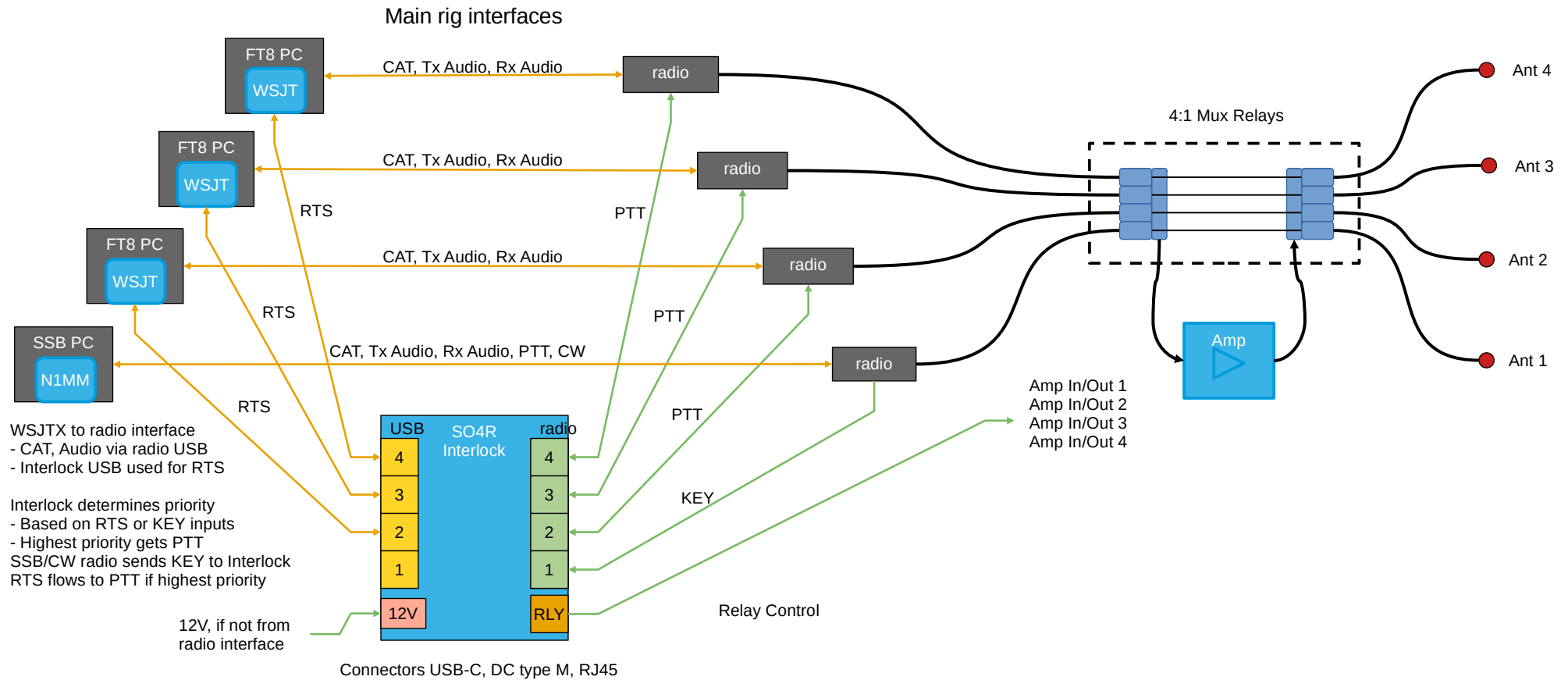


SO4R Interlock, V3

System Engineering: Fred Lass, K2TR
Design: Jeff Millar, WA1HCO

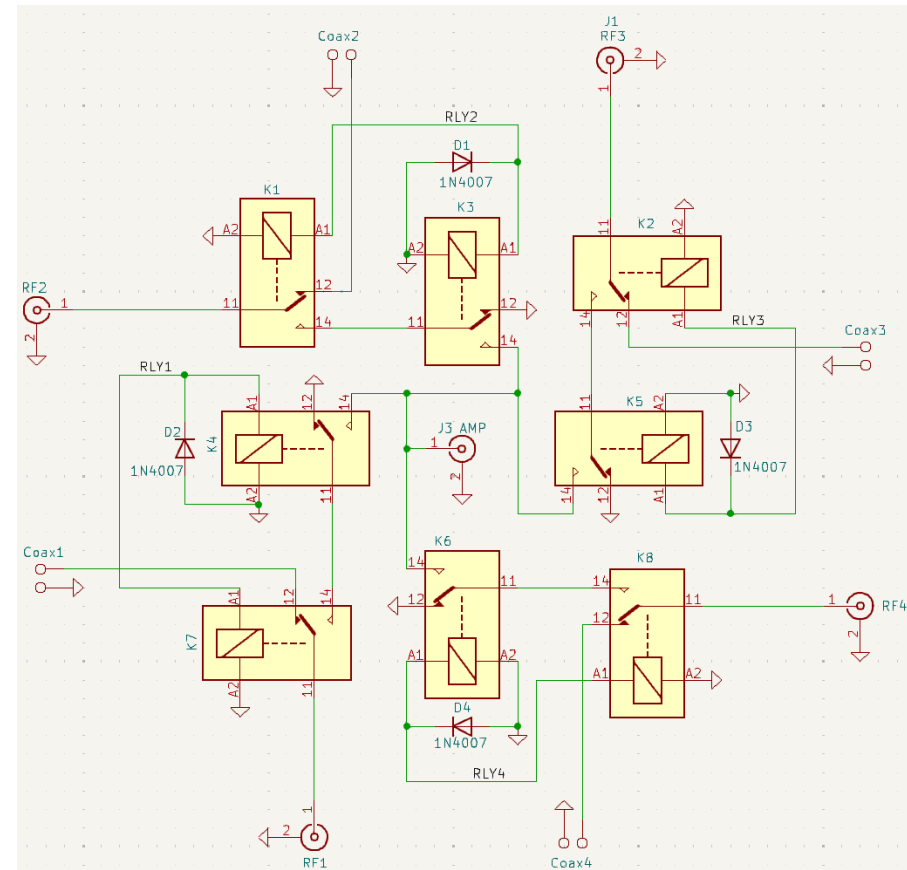


Baseline SO4R Interlock System Diagram



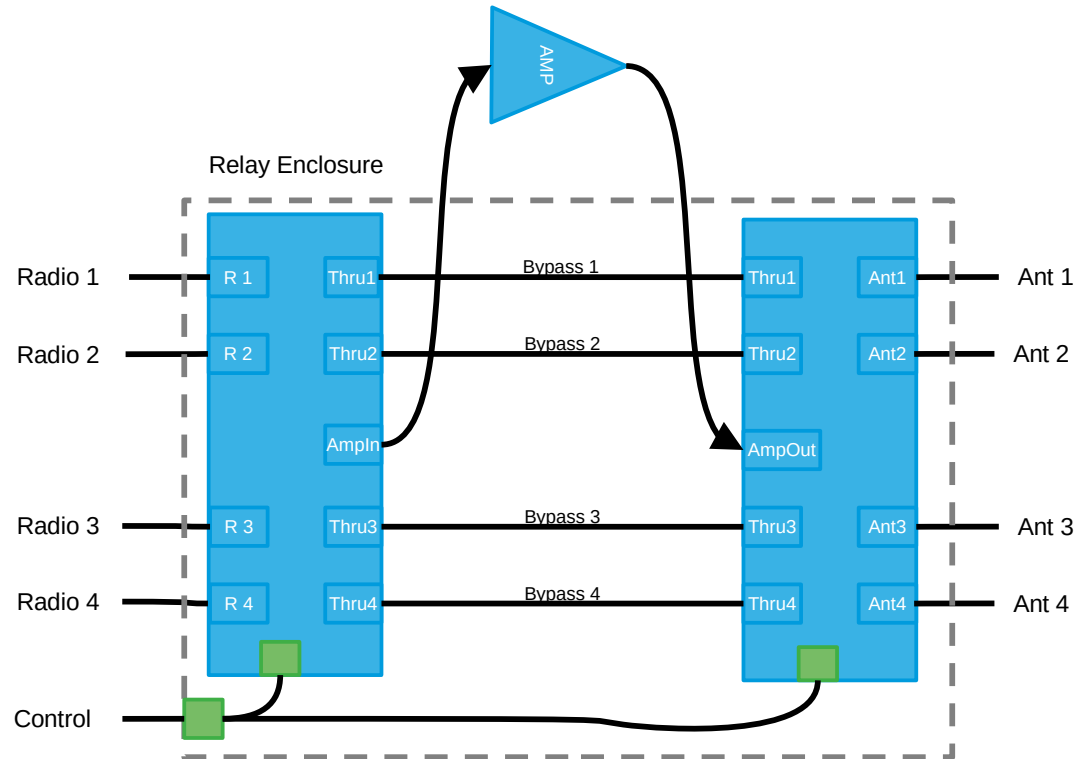
4:1 Mux with Bypass and Enhanced Isolation

- Two 4:1 Mux used
 - Amplifier input, amplifier output
- Functions
 - 4:1 Multiplexer
 - Radio to Amplifier if selected
 - Radio to Coax if not selected
 - Enhanced isolation
 - Isolation relays add grounded path between Amp and Receive paths



SO4R RF Diagram, V2

- 4 radios, 4 antennas
- 1 radio Tx at a time
- Controller Tx by priority
- Radio with Tx use amplifier
- Other radios use antenna



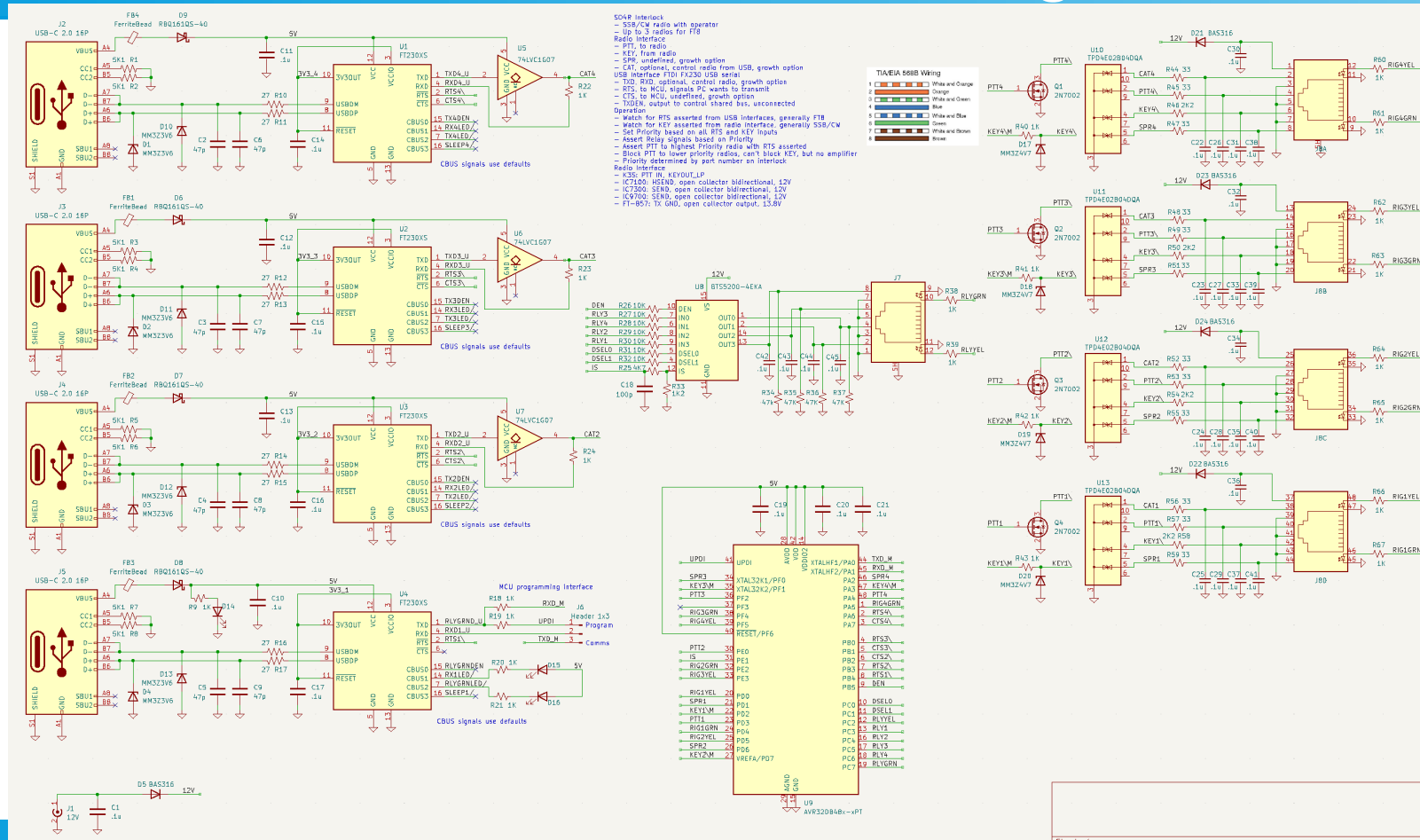
SO4R Interlock Principles of Operation

- Interlock for four radios, ensures only one Tx at a time
 - Read RTS signals from WSJTX Radios
 - Read KEY signal from SSB/CW Radios
 - Determine Priority from RTS and KEY inputs
 - Assert PTT to the highest priority WSJT radio if no SSB/CW key
 - Repeat every 20 to 50 usec
- SSB/CW radio has highest priority
 - Tx is controlled by VOX, Footswitch, CW key, etc. Not by Interlock
 - KEY from SSB/CW radio signal tells Interlock to block PTT to other radios
- Interlock forwards WSJTX RTS signal to radio PTT

SO4R Interlock Hardware Design

- Connectors
 - FT230X USB serial interface
 - RTS signal routed to microcontroller
 - Serial TX, RX converted to CAT, routed to radio
 - Radio interface
 - RJ45, 4 twisted pair of signal and ground
 - PTT output, open collector, 60V, 300 mA
 - KEY input, open collector, external pullup, protected to 15V
 - CAT in/out, open collector, 5V (growth option)
 - 12V, input from Rig
 - Relay Interface, RJ45, 4 twisted pair, relay and ground
 - Drive 12V to external relays
 - Relay driver has protection and diagnostics
 - Drive three Omron G2RL-1-E-12V, 125 mA total
 - 12V, DC power for relays, if no radio 12V
- AVR32DB48 microcontroller
 - Programmed using port 1 USB serial
 - Green LED for 5V power
 - White LEDs for serial traffic
- Electrical design
 - EMI filtering on all I/O
 - ESD protection on all inputs
 - 4 layer PCB
 - Three flooded ground planes
 - One flooded 5V plane
 - Interlock powered from any USB
 - Relays powered from any Radio or 12V coaxial

Interlock Schematic Diagram



SO4R Interlock Functional Design

- AVR32DB48 MicroController
 - Reads inputs, RTS from USB, KEY from radios
 - Writes outputs, PTT, Relay, and indicator LEDs
- Combinatorial logic with latency below 100 usec
- Relay protection relies on Radio Tx delay
- Radio timing assumptions
 - On PTT asserted, Radio asserts KEY output from radio immediately (usec)
 - On PTT asserted, Radio delays RF output 15 msec or more
 - PTT release causes KEY output to be released immediately (usec)
 - On Relay released, contacts take some msec to break connection
 - PTT release causes RF to stop in about 300 usec (filter delays)
- Conclusion, no need for sequencing within Interlock

SO4R Interlock LEDs

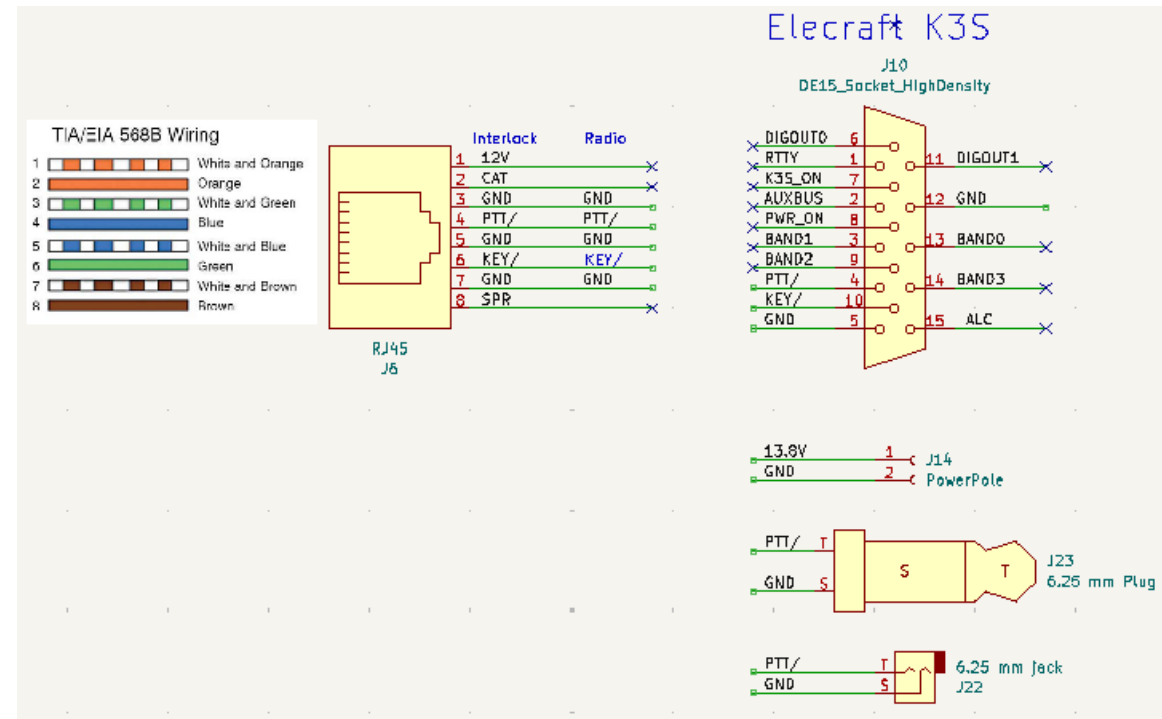
- Each RJ45 connector has a green and a yellow LED
- Radio interface
 - Green LED means radio is transmitting
 - RTS asserted, highest priority, PTT asserted, Relays asserted
 - Yellow LED means radio wants to transmit but blocked
 - RTS asserted, not highest priority, PTT blocked by Interlock
 - KEY asserted because operator pressed Transmit on radio
 - Radio will go into transmit
 - Interlock does not assert relays
 - Radio transmits through transfer relay to it's antenna

Corner Case

- Operator presses TX button on WSJT radio
 - LP radio _will_ go into transmit
 - Radio asserts KEY output
 - Interlock reads KEY and lights yellow LED on Radio RJ45
 - Interlock does not assert relays, Tx power flows to antenna not amplifier
- If SSB/CW radio already transmitting
 - Relays set to SSB/CW radio uses amplifier
 - WSJT Radio transfer relays remain in Radio to antenna mode
 - LP Tx power flows to antenna
 - LP RJ45 shows Yellow LED on Interlock
- If HP radio starts Tx while LP Radio in Tx
 - Interlock sees KEY from HP radio
 - Interlock asserts relays for HP radio
 - LP radio RJ45 remains yellow
 - HP radio RJ45 shows green

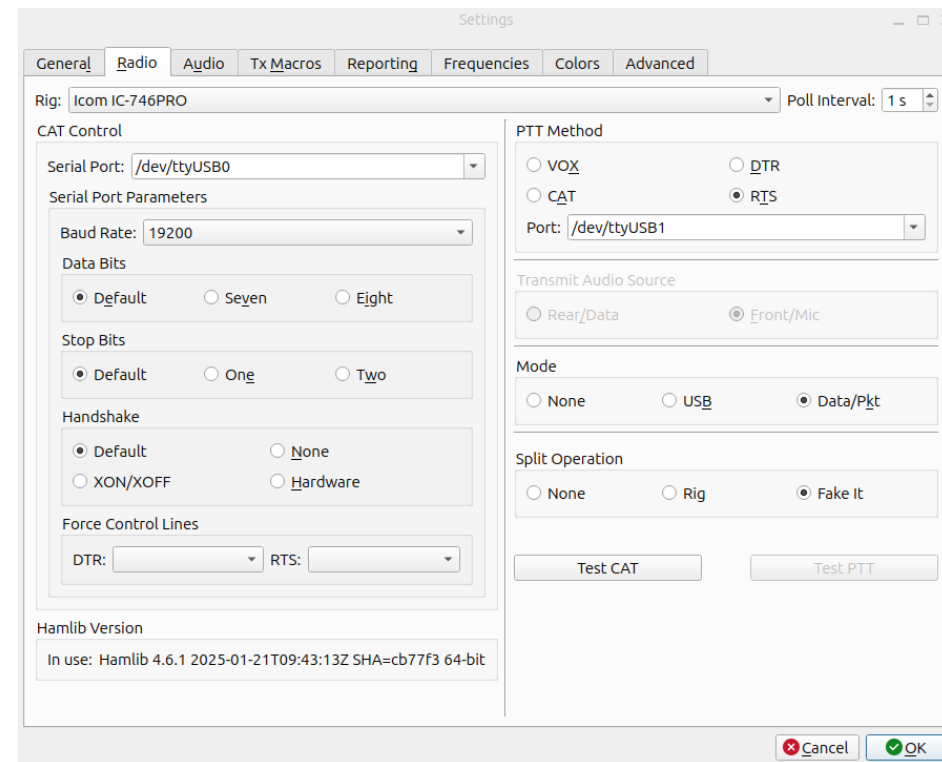
Interlock to Radio Interface Cables

- Make Cable
 - Use CAT 5/6 Ethernet cable
 - Cut in half to make two cables with RJ45
 - Expose 4 twisted pairs, strip ends
 - Solder on Radio interface connectors
 - Strain Relief the junction
- Sample schematic on the Radioht
 - Not based on experience with K3
 - Use KEY/, PTT/, and GND on DE15
 - Show ¼ inch phone jack and plug for PTT (optional)
 - Pick up 13.8V (optional)



WSJTX USB Interface from PC

- Separate USB interfaces
 - Linux WSJTX example shown
 - CAT Control USB
 - /dev/ttyUSB0
 - PTT Method USB set to RTS
 - /dev/ttyUSB1
- On Windows use separate COM ports for CAT and RTS



Identifying Interlock USB ports

- USB interfaces on Interlock programmed to make it easier to identify on a PC
 - FT230X EEPROM has reprogramed ID _strings_
 - Manufacturer: “FTDI WA1HCO”
 - Product: “SO4R Port n” where n = {1, 2, 3, 4}
- CAUTION: USB has TWO sources of Manufacturer and Product info
 - Hex codes: Vendor ID (VID) = 0x0403, Product ID (PID) = 0x6015
 - Strings: Manufacturer = FTDI WA1HCO, Product = FT230X SO4R PORT 3
 - Device Manager translates VID and PID into default string for device
 - Doesn't show the real strings within the device
 - On Windows, use USBTreeView for complete information about USB devices
 - Show programmed strings, used to find COM port number
 - https://www.uwe-sieber.de/usbtreetview_e.html

SO4R Interlock Options and Growth

- Radio 2, 3, 4 RJ45 connector has growth options
 - USB Serial Rx, Tx signals converted to CAT open collector
 - Spare input/output on Radio connector, direct to MCU, not protected
- Radio 1 RJ45 has options for serial interface
 - Serial port used for programming MCU
 - Serial port also available for user interface
- Relay driver, BTS4200-4 is connected to MCU
 - Fault detection, open, short, etc.
 - Measure relay currents

Interlock Code

```
void loop() {
  int Priority;

  // Read inputs, invert from low true to high true logic jeff millar, 4 weeks ago
  // RTS1 not used to prevent keying on programming or serial I/O
  bool RTS2 = !digitalRead(RTS2nPIN); // KEY from radio says Tx from RTS or manual
  bool RTS3 = !digitalRead(RTS3nPIN);
  bool RTS4 = !digitalRead(RTS4nPIN);

  // KEY 2,3,4
  // Not used for relays to prevent sequencing error if operator presses TX on radio
  // Used to light yellow LED on radio interface
  bool KEY1 = !digitalRead(KEY1nPIN); // KEY from radio says Tx from RTS or manual
  bool KEY2 = !digitalRead(KEY2nPIN); // KEY from radio says Tx from RTS or manual
  bool KEY3 = !digitalRead(KEY3nPIN); // KEY from radio says Tx from RTS or manual
  bool KEY4 = !digitalRead(KEY4nPIN); // KEY from radio says Tx from RTS or manual

  // Compute Priority based on RTS and KEY inputs
  // If no Tx requests, remains at 0 (Rx)
  Priority = 0; // Rx mode
  if (RTS4) Priority = 4; // WSJT radio
  if (RTS3) Priority = 3; // WSJT radio
  if (RTS2) Priority = 2; // WSJT radio
  if (KEY1) Priority = 1; // SSB/CW radio

  // Output signals in order of timing criticality

  // Assert Tx to relays
  // Use priority set from RTS and KEY
  // Relays asserted if either RTS or KEY input and priority
  bool RLY1 = (Priority == 1);
  bool RLY2 = (Priority == 2);
  bool RLY3 = (Priority == 3);
  bool RLY4 = (Priority == 4);
  digitalWrite(RLY1PIN, RLY1);
  digitalWrite(RLY2PIN, RLY2);
  digitalWrite(RLY3PIN, RLY3);
  digitalWrite(RLY4PIN, RLY4);

  // Assert PTT to radios with RTS and corresponding to priority, all others stay Rx
  // PTT asserted only on RTS and priority, do not use KEY
  // SSB/CW radio doesn't need PTT from Interlock
  bool PTT2 = RTS2 and (Priority == 2);
  bool PTT3 = RTS3 and (Priority == 3);
  bool PTT4 = RTS4 and (Priority == 4);
  digitalWrite(PTT2PIN, PTT2);
  digitalWrite(PTT3PIN, PTT3);
  digitalWrite(PTT4PIN, PTT4);

  // Assert Rig RJ45 Green LED if rig is transmitting
  digitalWrite(RIG1GRNPIN, RLY1);
  digitalWrite(RIG2GRNPIN, RLY2);
  digitalWrite(RIG3GRNPIN, RLY3);
  digitalWrite(RIG4GRNPIN, RLY4);

  // Assert Rig RJ45 Yellow LED if
  // WSJT asking for Tx but not highest priority
  // Radio indicates KEY without RTS, meaning operator pressed TX button
  bool INH2 = (RTS2 and (Priority > 0) and (Priority < 2)) or (KEY2 and !RTS2);
  bool INH3 = (RTS3 and (Priority > 0) and (Priority < 3)) or (KEY3 and !RTS3);
  bool INH4 = (RTS4 and (Priority > 0) and (Priority < 4)) or (KEY4 and !RTS4);
  digitalWrite(RIG2YELPIN, INH2);
  digitalWrite(RIG3YELPIN, INH3);
  digitalWrite(RIG4YELPIN, INH4);

  // Assert Relay RJ45 Green LED if any rig transmitting
  // Assert Relay RJ45 Yellow LED if any rig inhibited
  digitalWrite(RLYGRNPIN, (Priority > 0));
  digitalWrite(RLYYELPIN, INH2 or INH3 or INH4);
}
```

Github Repository

- Project has multiple branches, main, V1.1, V2, and V3
 - Use V3 as of 1/26
 - https://github.com/wa1hco/SO4R_Interlock/tree/V2
- Within a branch, several directories
 - Hardware, used for Kicad schematics, components, documentation
 - SO4R_Interlock
 - Radio_Interface_Cables
 - Transfer_G2RL
 - Software
 - vscode, PlatformIO, Arduino Framework
 - Readme.md